ME3101: Data Analytics and Machine Learning for Engineers

Project Phase II: Classification of Metal Surface Defects

Team Members Name**:**

GAJENDRA (2301ME18)

MOHIT MEENA (2301ME34)

MOHIT SAHU (2301ME35)

PRIYANSHU PRIYAM (2301ME41)

SAI KUMAR MALAVATH (2303ME14)

Part 1: Data Loading & Visualization

Dataset Overview

The dataset provided for this project consists of grayscale images of hot-rolled metal sheets, categorized by surface defects. The objective is to classify these images into one of six defect categories:

1. Crazing
2. Inclusion
3. Patches
4. Pitted
5. Rolled
6. Scratches

The data is pre-split into three main directories: train, valid (validation), and test. Each of these directories contains 6 sub-folders, one for each defect class, holding the corresponding images.

Data Loading and Preprocessing

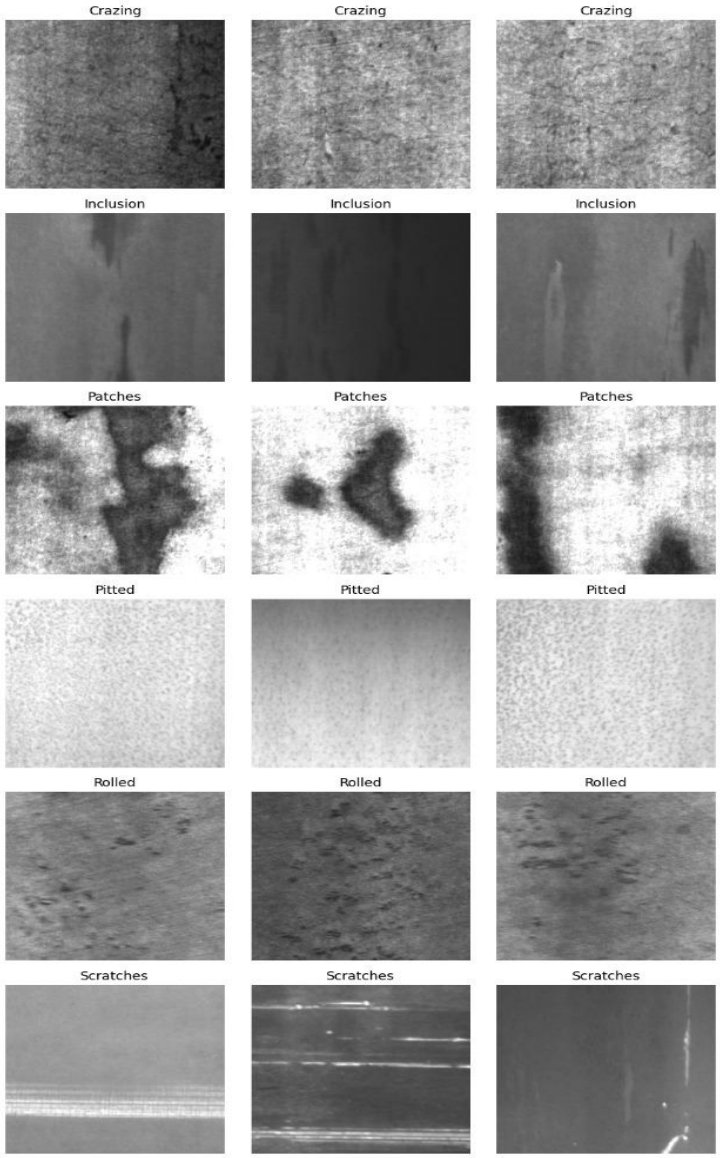
To prepare the data for the Convolutional Neural Network (CNN), the following preprocessing steps were taken:

- **Image Loading:** Images were loaded from their respective directories using Keras's Image Data Generator (or image_dataset_from_directory).

- **Resizing:** All images were resized to a uniform dimension of [e.g., 200x200] pixels to ensure consistent input size for the model.
- **Normalization:** Pixel values, originally in the range [0, 255], were normalized by dividing by 255. This aids in faster and more stable model training.
- **Label Encoding:** The 6 categorical class names (Crazing, Inclusion, etc.) were converted into one-hot encoded vectors, which is the required format for categorical cross-entropy loss.

Data Visualization

To understand the visual characteristics of each defect, sample images from each of the 6 classes in the training set are shown below.



Part 2: Build a Simple CNN Model

Baseline Model Architecture

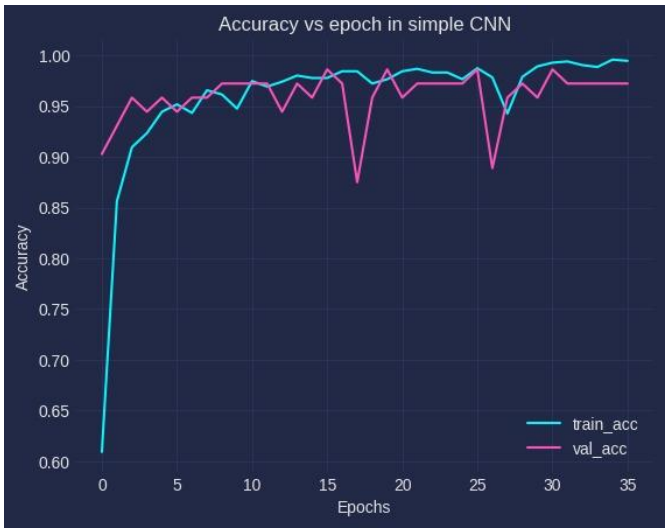
A baseline CNN model was constructed to serve as a benchmark. The architecture is sequential and consists of:

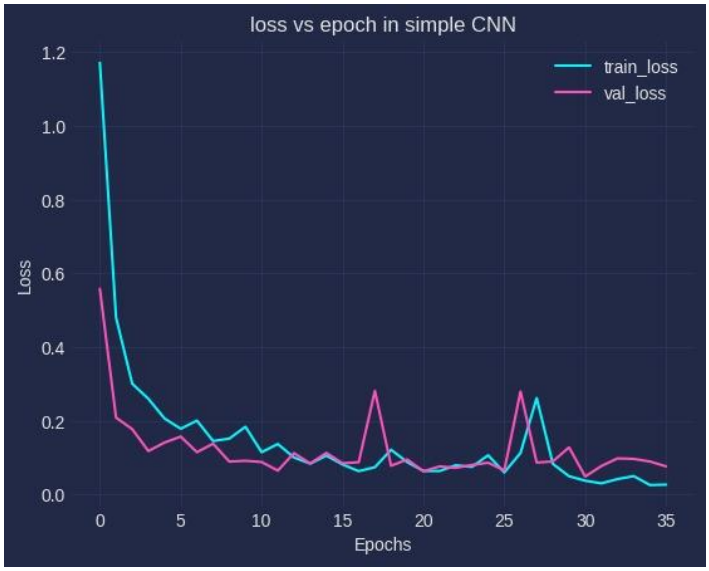
1. A Conv2D layer with [32] filters, a (3,3) kernel, and relu activation.
2. A MaxPooling2D layer with pool size (2,2).
3. A Conv2D layer with [64] filters, a (3,3) kernel, and relu activation.
4. A MaxPooling2D layer with pool size (2,2).
5. A Flatten layer to convert the 2D feature maps into a 1D vector.
6. A Dense (fully connected) layer with [128] units and relu activation.
7. A final Dense output layer with 6 units (one for each class) and softmax activation.

The model was compiled using the Adam optimizer and categorical_crossentropy as the loss function.

Baseline Model Performance

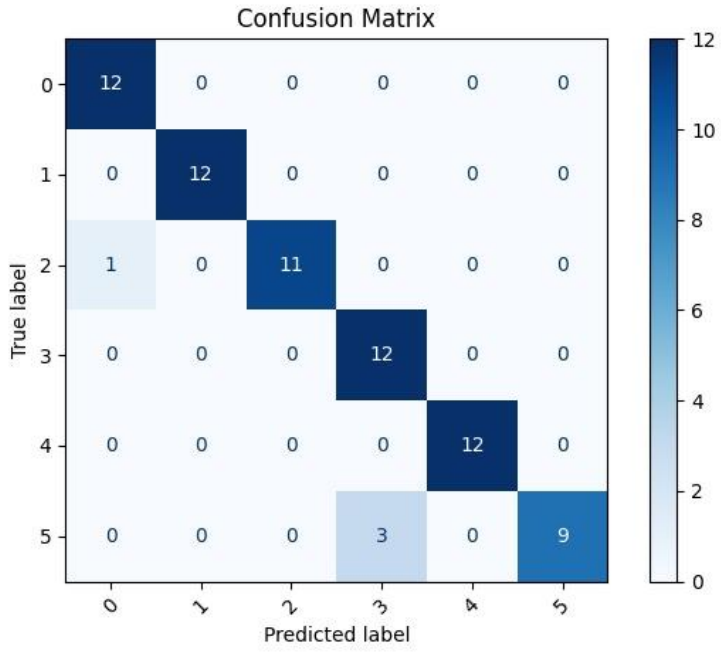
The model was trained for [100] epochs with early stopping based on validation accuracy.





Baseline Results:

Metric	Value
Best Validation Accuracy	0.9861
Test Loss	0.1357
Test Accuracy	0.9583
Precision	0.9583
Recall	0.9615
F1-Score	0.9582



Part 3: Add Improvements

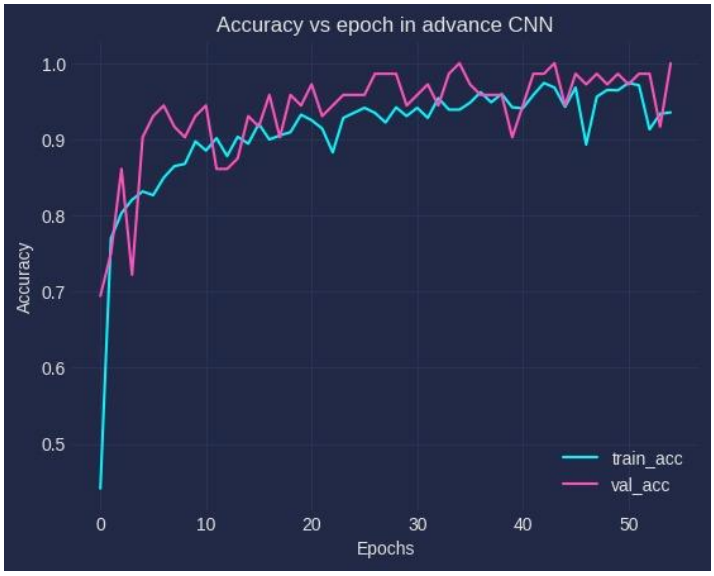
To address the overfitting observed in the baseline model and improve generalization, a second model was built incorporating Data Augmentation, Dropout, and Batch Normalization.

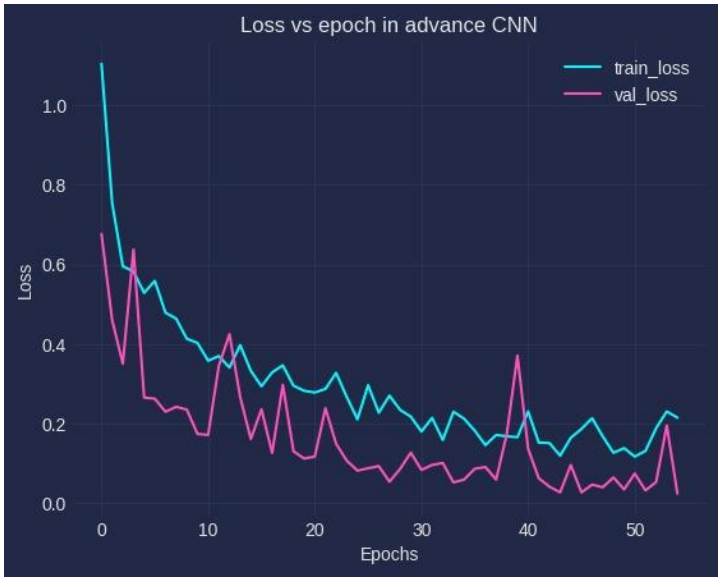
Improved Model Architecture & Techniques

1. **Data Augmentation:** The ImageDataGenerator was configured to apply random transformations to the training images in real-time. These included:
    - Rotation ([40 degrees])
    - Width, Height Shift ([0.1], [0.1])
    - Zoom ([0.7])
    - Horizontal Flip
2. **Dropout:** [0.3, 0.2, 0.5]
3. **Batch Normalization:** BatchNormalization layers were added after Conv2D layers (before activation).

Improved Model Performance

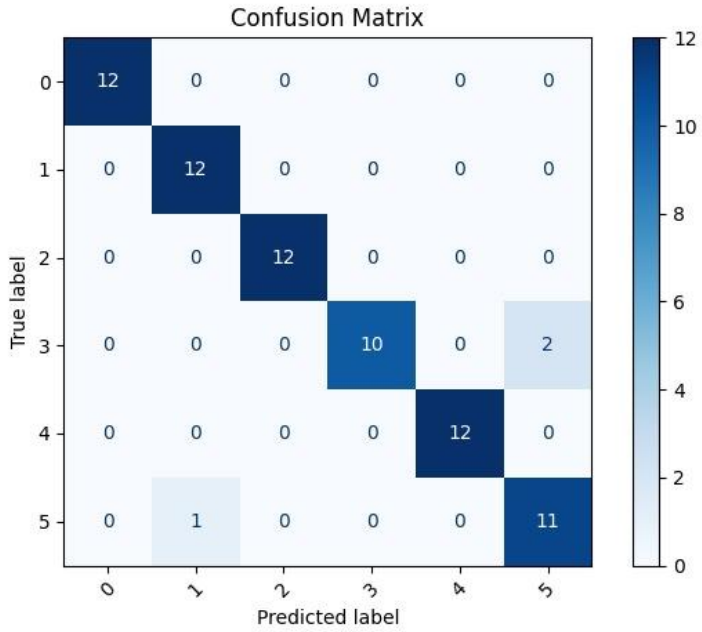
The model was trained for [100] epochs with early stopping based on validation accuracy.





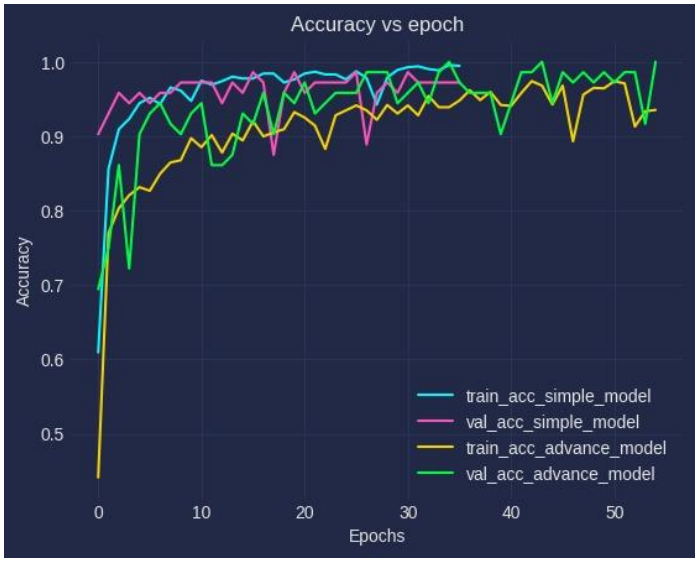
Improved Results:

Metric	Value
Best Validation Accuracy	**1.0**
Test Loss	**0.2102**
Test Accuracy	**0.9444**
Precision	**0.9538**
Recall	**0.9444**
F1-Score	**0.9438**



Part 4: Report and Discussion

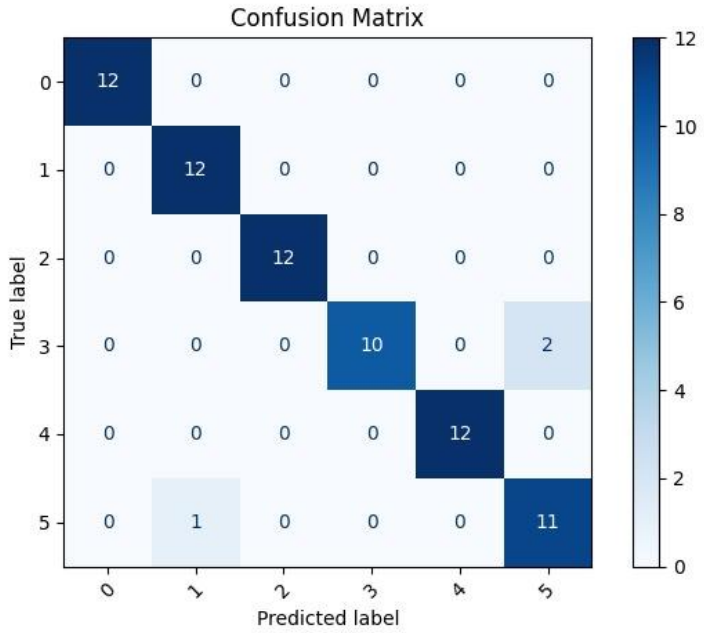
Model Comparison



Metric	Baseline CNN	Improved CNN
Validation Accuracy	[98.61%]	[100%]
Test Accuracy	[95.83%]	[94.44%]
Loss	[0.1357]	[0.2102]

Confusion Matrix (Best Model)

The confusion matrix for the **Improved CNN Model** on the test dataset is shown below.



Discussion

- **Which improvements helped most?** Data Augmentation likely had the largest impact. By creating new, varied training samples, it forced the model to learn the underlying features of each defect rather than memorizing specific images.
- **Presence of Overfitting:** There is very little overfitting in the baseline model, however including regularization techniques helped to create more complex model so that it can learn from more data which we generated from augmentation techniques.
- **Model Generalization:** The improved model's ability to generalize to unseen defect patterns is significantly better because it has seen more data that came from augmentation.

Conclusion and Future Improvements

This project successfully demonstrated the development of a CNN for classifying metal surface defects.

Potential future improvements could include:

- **Transfer Learning:** Using a pre-trained model (e.g., VGG16, ResNet50) and fine-tuning it on this dataset could yield even higher accuracy.

- **Hyperparameter Tuning:** A more systematic search for optimal parameters (e.g., dropout rate, learning rate, number of layers) could further enhance performance.
- **Deeper CNN:** A deeper custom architecture could potentially learn more complex features.